THE HUNGRY HELM — VIVA VOCE DEFENSE & DEMONSTRATION GUIDE

Agile Software Development (CSET455) | Evaluator: Rashmi Rani | Student: Prikshit Sharma (S24CSEU1823)

*This cheat sheet prepares you for rapid-fire technical and conceptual questioning during tomorrow's evaluation. Every answer connects abstract Agile principles directly to your real Jira project (**THH**) and your live verified codebase (ports 5001 & 3000).*

Part 1: Agile, Scrum & Requirements Engineering (Labs 1 & 2)

Q1: Why did you choose Scrum over Kanban for The Hungry Helm?

Direct Answer: The Hungry Helm is an iteration-driven product development effort with clear time-boxed milestones (Sprints 1 to 4) and cross-functional feature dependencies (auth, ordering, kitchen dispatch). Scrum's sprint planning, sprint goals, and fixed cadence provide the predictability needed for delivering a cohesive v1.0 release. Kanban is better suited for continuous support or IT helpdesk ticket streams (as in Lab 4).

Q2: What is the INVEST criteria, and how do your User Stories meet it?

Direct Answer: INVEST stands for **Independent, Negotiable, Valuable, Estimable, Small, and Testable**. In The Hungry Helm, each of the 14 stories is vertical and independently deliverable. For example, THH-8 (Server-Side Price Verification) is Small (5 SP), Valuable to security, and Testable via explicit Given/When/Then acceptance criteria: *'Given a cart with tampered item price ■1, when checkout is submitted, backend fetches DB price ■180 and calculates verified total.'*

Q3: How do you distinguish an Epic from a User Story?

Direct Answer: An **Epic** represents a substantial body of work or broad capability that cannot be completed within a single sprint and must be decomposed (e.g. THH-E4: Cart Management & Tamper-Proof Order Placement). A **User Story** is a thin, vertical slice of user value sized to fit comfortably within a two-week sprint (e.g. THH-7: Persistent Cart = 5 SP, and THH-8: Server Checkout = 5 SP).

Q4: Why do you use Story Points and Fibonacci numbers instead of hours or days?

Direct Answer: Estimating in hours falls victim to developer optimism, individual skill variations, and false precision. **Story Points** represent relative sizing—factoring in complexity, uncertainty, and effort. We use the modified Fibonacci sequence (1, 2, 3, 5, 8, 13) because uncertainty scales non-linearly; it is easier to agree whether a task is twice as hard as another than whether it takes 14 versus 16 hours.

Part 2: Estimation, Velocity Planning & MoSCoW Scope Trade-Offs (Labs 3, 5, 6)

Q5: What was your agreed baseline anchor story, and why?

Direct Answer: Our agreed baseline anchor is THH-1: Multi-Outlet Seed & Campus Data Model, sized at **3 Story Points**. It was chosen because seeding 5 campus stalls in Mongoose involves well-understood schema structures, standard JSON fixtures, and zero unknown external dependencies. Every other story was sized relative to this 3-point benchmark.

Q6: How did you calculate team capacity and plan your 4 Sprints?

Direct Answer: We calibrated a team velocity of **20 Story Points per 2-week Sprint**. Across our 67 total points, we sequenced stories based on technical dependencies across 4 Sprints: Sprint 1 (18 pts - Core Platform), Sprint 2 (18 pts - Ordering & Security), Sprint 3 (18 pts - Kitchen Display), and Sprint 4 (13 pts - Hardening & Release). No sprint exceeded the 20-point limit.

Q7: How would you respond if the client advanced the launch date by one week?

Direct Answer: In traditional Waterfall, project managers attempt 'crashing' (adding overtime) or 'fast-tracking' (running tasks in parallel), which introduces severe quality bugs. In Agile, we negotiate **Scope Trade-Offs using MoSCoW prioritization**. When simulating an accelerated deadline, we deferred non-essential features (Razorpay online payment, 8 pts, and FCM push notifications, 5 pts) into Phase 3 (Won't Have v1.0). This protected our core ordering and kitchen dispatch pipeline without compromising quality.

Q8: What is a Hardening Sprint, and why is Sprint 4 sized at 13 points instead of 20?

Direct Answer: A **Hardening Sprint** (Lab 6 Task 4) is an iteration reserved purely for stabilization, end-to-end integration testing, performance verification, and security audits—no new feature stories are introduced. Sprint 4 was intentionally sized to 13 planned points (THH-12, THH-13, THH-14), leaving a 7-point contingency buffer (~35%) for addressing unexpected bugs discovered during pre-submission testing.

Part 3: Custom Jira Workflows, Statuses & Quality Gates (Lab 7)

Q9: What custom workflow statuses and rules did you configure in Jira?

Direct Answer: We engineered the THH_Software_Scrum_Workflow with 5 distinct statuses: **Backlog** → **To Do** → **In Progress** → **In Review / QA** → **Done**, plus a reverse defect loop from In Review back to In Progress.
We enforced three types of transition rules:
• **Condition:** Only users in the 'Developers' or 'Administrators' project role can execute transitions.
• **Validator:** On the 'Approve & Complete' transition, the fields Resolution and Fix Version (v1.0) must not be empty, and the linked GitHub PR must be merged.
• **Post-Function:** Automatically sets Resolution to 'Completed' and fires an 'Issue Resolved' email event to the Project Lead.

Q10: What is the difference between a Condition, a Validator, and a Post-Function in Jira?

Direct Answer:
• **Condition:** Evaluated *before* a user sees a transition button. If false, the button is completely hidden.
• **Validator:** Checked *after* the user clicks the transition button and enters screen data. If validation fails, an error message appears and the issue does not move.
• **Post-Function:** An automated system action executed *immediately after* a successful transition (e.g. setting fields, clearing assignees, sending webhooks).

Part 4: Agile Metrics, Burndown, Burnup & Dashboards (Labs 4, 6, 8)

Q11: How do you interpret a Sprint Burndown chart vs a Release Burnup chart?

Direct Answer:
• **Sprint Burndown:** Plots *remaining Story Points against time (working days)* in a single sprint. If the line is above the guideline, the team is behind; if below, ahead. On Day 6 of our Sprint 2, an upward bump shows a +3 pt scope trade-off that was burned down by Day 9.
• **Release Burnup:** Plots *completed points against total scope over multiple sprints* toward a release (v1.0). It clearly reveals whether delays are caused by slow delivery or expanding scope (scope creep).

Q12: What does a Cumulative Flow Diagram (CFD) tell you about a team's workflow?

Direct Answer: The CFD shows the cumulative count of issues in each status over time. Parallel bands with a steady upward slope indicate a stable, predictable process. A widening 'In Progress' band indicates multitasking bottlenecks and violating WIP limits. In Hungry Helm, our CFD shows an average Work-In-Progress of 2.1 stories, Lead Time of 4.2 days, and Cycle Time of 2.1 days.

Q13: What gadgets did you place on your Jira Agile Dashboard?

Direct Answer: We placed 4 specialized gadgets: (1) **Sprint Burndown Gadget** for live iteration tracking, (2) **Created vs. Resolved Chart** showing 20 created and 20 resolved (100% resolution), (3) **Two-Dimensional Filter Statistics** cross-tabulating status vs priority, and (4) **Velocity Chart** showing commitments vs actual completions across all 4 sprints.

Part 5: Hungry Helm Full-Stack Architecture & Implementation

Q14: How does your application connect to MongoDB, and how did you solve connection failures?

Direct Answer: In backend/server.js, we engineered an architectural fallback using mongodb-memory-server. When the server boots, it first attempts to connect to MongoDB Atlas using process.env.MONGO_URI. If network restrictions, captive portals, or firewall rules block Atlas, the catch block automatically spins up a local in-memory MongoDB instance and triggers seedDatabase() immediately. This guarantees that evaluators can run the system anywhere with zero database configuration.

Q15: How does your server prevent client-side price tampering during order checkout?

Direct Answer: In backend/controllers/orderController.js, the backend **never trusts prices sent in the client request body**. Instead, it extracts the outletId and items[i].itemId, queries the database source-of-truth for the canonical item prices, recalculates the subtotal, taxes, and grand total on the server, and persists the verified amount. If a malicious user intercepts the HTTP payload and edits ■180 to ■1, the server overwrites it with ■180.

Q16: How is role-based authentication enforced between students and outlet staff?

Direct Answer: In backend/middleware/auth.js, incoming requests pass through JWT verification. The token payload encodes the user's role (student vs staff) and for staff, their specific outletId (e.g. ik for Kathi Castle). When staff access the Kitchen Display System (/api/orders/staff), the controller enforces a query filter { outlet: req.user.outletId }, strictly isolating each kitchen's tickets.

Q17: Why did you shift the backend port from 5000 to 5001?

Direct Answer: On macOS (Sonoma/Sequoia), the native Control Center / AirPlay Receiver service listens on port 5000 by default (EADDRINUSE: :::5000). In THH-12, we updated backend/.env to PORT=5001 and modified frontend/script.js to target http://localhost:5001/api, ensuring clean zero-conflict execution.

Part 6: Step-by-Step Live Demonstration Script for Faculty

Step 1: Open Running Services
Open browser tabs: Frontend at http://localhost:3000 and Backend Health at http://localhost:5001/api/health. Show that both daemons are active and healthy.

Step 2: Demonstrate Student Authentication & Food Chart (THH-3, THH-5)
Log in as prikshit@bennett.edu.in with password password. Point out the nautical campus chart displaying live operational badges ('Open', '15-20 min wait') for Kathi Castle, Southern Soul, etc.

Step 3: Demonstrate Cart & Single-Outlet Conflict Shield (THH-6, THH-7)
Click 'Kathi Castle' → Add 'Paneer Kathi Roll' (■180). Open cart. Then browse 'Southern Soul' and attempt to add 'Masala Dosa'—show the modal prompt explaining that orders cannot cross multiple kitchens.

Step 4: Demonstrate Tamper-Proof Order Placement (THH-8)
Click Checkout. Show the console network tab: order #101 is created for ■300 (■180 + ■120 mock additions) with server-calculated totals. Point out the Voyage Log displaying status 'incoming'.

Step 5: Demonstrate Staff Kitchen Display & State Machine (THH-9, THH-10)
In a separate window/incognito, open http://localhost:3000/kds.html. Log in as IK-STAFF-001 / staff123. Show order #101 appearing on the KDS board. Advance status: incoming → preparing → ready.

Step 6: Show Synchronized Student Voyage Log Update (THH-11)
Switch back to the student tab. Demonstrate the animated compass timeline lighting up to 'Food is Ready for Pickup!'. Show how polling synchronized the state seamlessly.

Step 7: Show Jira Project & GitHub DoD (THH-14, Lab 6)
Open Jira project THH on prikshittt777.atlassian.net. Show the 4 Sprints in the Backlog, the active Scrum board in Done status, the Custom Workflow in Project Settings, and the GitHub commit linkage in the Development panel.